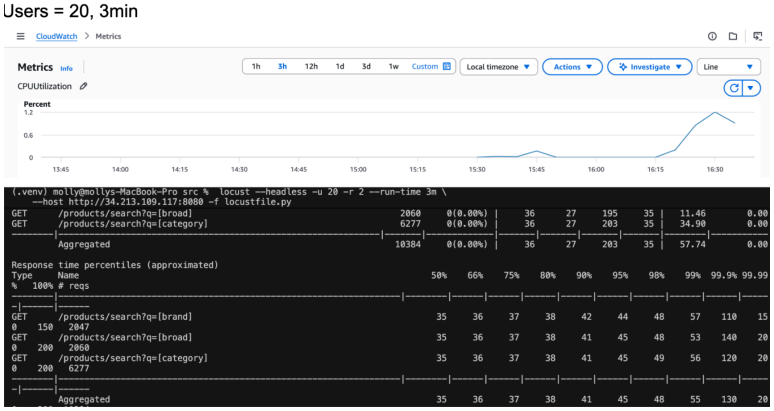
HW6 Report

Part 2: Identifying Performance Bottlenecks (Single Instance)

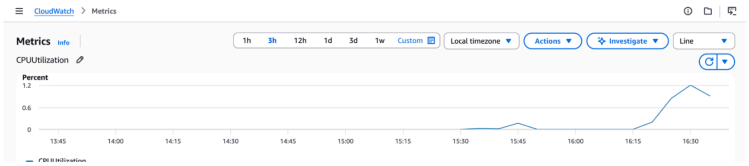
1. Stress Test Data (Baseline)

User Count	Latency (50%/95%/99%)	Max RPS	CPU Avg	Status
5 Users	36 / 45 / 57 ms	14.56	~0.3%	Healthy
20 Users	35 / 45 / 55 ms	57.74	~1.2%	Healthy
50 Users	38 / 53 / 120 ms	136.14	27.63%	Load Increasing
100 Users	43 / 77 / 170 ms	247.34	~49%	Degrading
200 Users	89 / 310 / 510 ms	344.55	~53.38%	Breaking Point

Users = 20, 3min

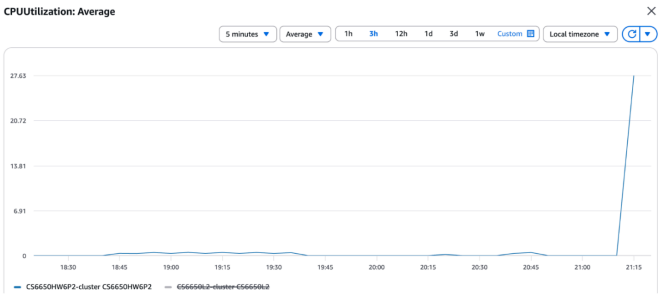


Users = 5, 2min



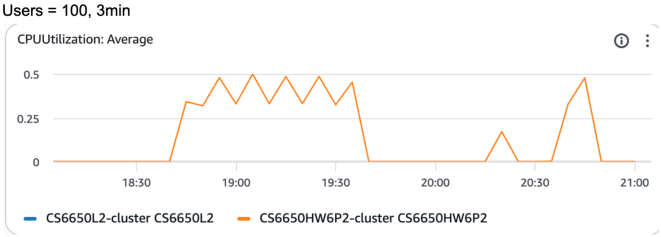

```
(.venv) molly@mollys-MacBook-Pro src %  locust --headless -u 5 -r 1 --run-time 2m \
    --host http://34.213.109.117:8080 -f locustfile.py
GET      /products/search?q=[brand]                                  370     0(0.00%) |     36      28      82      36 |    3.09        0.00
GET      /products/search?q=[broad]                                  339     0(0.00%) |     36      28      81      35 |    2.83        0.00
GET      /products/search?q=[category]                              1036     0(0.00%) |     36      28     159      36 |    8.64        0.00
--------|----------------------------------------------------------|-------|-------------|-------|-------|-------|-------|--------|-----------
         Aggregated                                                1745     0(0.00%) |     36      28     159      36 |   14.56        0.00

Response time percentiles (approximated)
Type     Name                                                          50%    66%    75%    80%    90%    95%    98%    99%  99.9% 99.99
%   100% # reqs
--------|----------------------------------------------------------|--------|------|------|------|------|------|------|------|------|-----
-|-------|-------
GET      /products/search?q=[brand]                                     36     37     38     39     42     46     52     64     82      8
2     82    370
GET      /products/search?q=[broad]                                     35     36     37     38     41     46     51     58     82      8
2     82    339
GET      /products/search?q=[category]                                  36     37     38     38     41     44     49     56    110     16
0    160   1036
--------|----------------------------------------------------------|--------|------|------|------|------|------|------|------|------|-----
-|-------|-------
         Aggregated                                                     36     37     38     38     41     45     50     57    110     16
0    160   1745
```

Users = 50, 3min



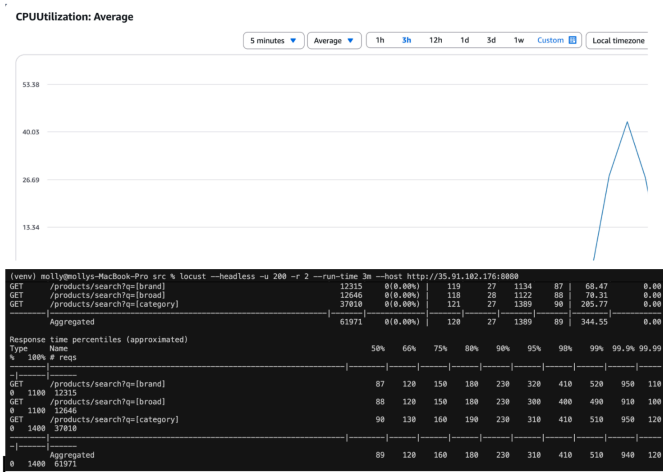

```
(venv) molly@mollys-MacBook-Pro src % locust --headless -u 50 -r 2 --run-time 3m --host http://35.91.102.176:8080
GET      /products/search?q=[brand]                                 4829     0(0.00%) |     40      28     247      38 |   26.85        0.00
GET      /products/search?q=[broad]                                 4930     0(0.00%) |     40      28     238      38 |   27.42        0.00
GET      /products/search?q=[category]                             14722     0(0.00%) |     40      28     320      38 |   81.87        0.00
--------|----------------------------------------------------------|-------|-------------|-------|-------|-------|-------|--------|-----------
         Aggregated                                                24481     0(0.00%) |     40      28     320      38 |  136.14        0.00

Response time percentiles (approximated)
Type     Name                                                          50%    66%    75%    80%    90%    95%    98%    99%  99.9% 99.99
%   100% # reqs
--------|----------------------------------------------------------|--------|------|------|------|------|------|------|------|------|-----
-|-------|-------
GET      /products/search?q=[brand]                                     38     41     42     43     47     54     85    130    210     25
0    250   4829
GET      /products/search?q=[broad]                                     38     41     42     43     47     54     87    130    220     24
0    240   4930
GET      /products/search?q=[category]                                  38     41     42     43     47     53     82    120    220     32
0    320  14722
--------|----------------------------------------------------------|--------|------|------|------|------|------|------|------|------|-----
-|-------|-------
         Aggregated                                                     38     41     42     43     47     53     84    120    220     29
```

Users = 100, 3min




```
(venv) molly@mollys-MacBook-Pro src % locust --headless -u 100 -r 2 --run-time 3m --host http://35.91.102.176:8080
GET      /products/search?q=[brand]                                 8934     0(0.00%) |     48      27     428      43 |   49.67        0.00
GET      /products/search?q=[broad]                                 8875     0(0.00%) |     48      28     388      43 |   49.34        0.00
GET      /products/search?q=[category]                             26676     0(0.00%) |     47      27     393      43 |  148.32        0.00
--------|----------------------------------------------------------|-------|-------------|-------|-------|-------|-------|--------|-----------
         Aggregated                                                44485     0(0.00%) |     47      27     428      43 |  247.34        0.00

Response time percentiles (approximated)
Type     Name                                                          50%    66%    75%    80%    90%    95%    98%    99%  99.9% 99.99
%   100% # reqs
--------|----------------------------------------------------------|--------|------|------|------|------|------|------|------|------|-----
-|-------|-------
GET      /products/search?q=[brand]                                     43     45     47     48     55     85    140    190    310     43
0    430   8934
GET      /products/search?q=[broad]                                     43     45     47     48     54     78    130    190    320     39
0    390   8875
GET      /products/search?q=[category]                                  43     45     47     48     53     74    120    160    300     38
0    390  26676
--------|----------------------------------------------------------|--------|------|------|------|------|------|------|------|------|-----
-|-------|-------
         Aggregated                                                     43     45     47     48     54     77    130    170    300     38
0    430  44485
```

Users = 200， r = 2, 3min



2. Bottleneck Analysis

- Explain what happened when load increased: As the load increased from 5 to 200 users, we observed a transition from linear performance to severe degradation. Below 50 users, the system scaled predictably with stable latencies. However, at 100 users, we noticed a "divergence" where P99 latency nearly doubled compared to P50. At the 200-user mark, the system hit a "performance wall": while RPS only increased slightly, the P99 latency spiked by 895% (reaching 510ms). This indicates the system could no longer process the incoming request queue in real-time.
- Evidence for Scaling vs. Optimization: I determined that this is a scaling issue rather than a code problem based on two key pieces of evidence:
  1. Constant Computational Complexity: The search logic is hard-coded to check exactly 100 products per request. Since the per-request work is fixed and thread-safe storage (sync.Map) is already implemented, there are no "low-hanging fruit" optimizations left in the algorithm.
  2. Resource Utilization Patterns: Memory usage remained completely flat, proving no leaks or excessive object allocation. Meanwhile, even though CPU Average sat at ~53%, the massive spike in tail latency confirms the 0.25 vCPU limit is insufficient for the high concurrency of 200 users. The bottleneck is the vCPU scheduling capacity, which can only be solved by adding more compute "lanes."
- CloudWatch Metrics for Scaling Decisions: Using CloudWatch metrics, I identified that 27% CPU (50 users) is the upper limit for "perfect" performance, while 50% CPU (100-200 users) leads to unacceptable latency. Scaling Decision: I will implement an Auto Scaling policy with a 40% CPU utilization target. This threshold ensures that the system triggers a scale-out event *before* reaching the 50% saturation point where user experience begins to degrade.
- Creative Stress Testing: I employed a Step-Load Stress Test with a low spawn rate (-r 2). Unlike a "flash" test, this gradual approach allowed me to pinpoint the exact saturation

threshold of the CPU scheduler. I also monitored the "Maximum" vs "Average" CPU metrics; the divergence between these two in CloudWatch provided the final proof that individual request spikes were overwhelming the single-core environment despite the average load appearing moderate.

Part 3: Horizontal Scaling Analysis (ALB + Auto Scaling)

Test 1: Comparison - Solving the Part II Bottleneck

Goal: Run the exact 200-user load from Part II against the ALB endpoint.

Metric	Part II (1 Task)	Part III (2+ Tasks)	Improvement
P99 Latency	510 ms	140 ms	~72.5%
Max RPS	344.55	413.08	~19.9%

Users = 200 r =2 3min

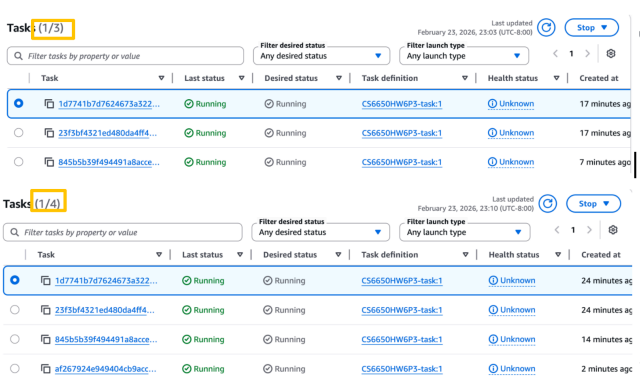
```
--------|----------------------------------------------------------------------|-------|-------------|-------|-------|-------|-------|--------|----------
GET     /products/search?q=[brand]                                             14796     0(0.00%) |    48     27    1181    45 |  82.27       0.00
GET     /products/search?q=[broad]                                             15120     0(0.00%) |    50     27    1462    45 |  84.07       0.00
GET     /products/search?q=[category]                                          44372     0(0.00%) |    49     27    1446    45 | 246.73       0.00
--------|----------------------------------------------------------------------|-------|-------------|-------|-------|-------|-------|--------|----------
        Aggregated                                                             74288     0(0.00%) |    49     27    1462    45 | 413.08       0.00

Response time percentiles (approximated)
Type     Name                                                                     50%    66%    75%    80%    90%    95%    98%    99%  99.9% 99.99
%   100% # reqs
--------|----------------------------------------------------------------------|--------|------|------|------|------|------|------|------|------|-----
-|------|------
GET     /products/search?q=[brand]                                                 45     46     48     48     51     59     95    130    790     90
0   1200  14796
GET     /products/search?q=[broad]                                                 45     46     48     49     52     62    110    150    850    150
0   1500  15120
GET     /products/search?q=[category]                                              45     46     48     48     51     61    100    140    770     92
0   1400  44372
--------|----------------------------------------------------------------------|--------|------|------|------|------|------|------|------|------|-----
-|------|------
        Aggregated                                                                 45     46     48     48     51     60    100    140    790    130
0   1500  74288
```

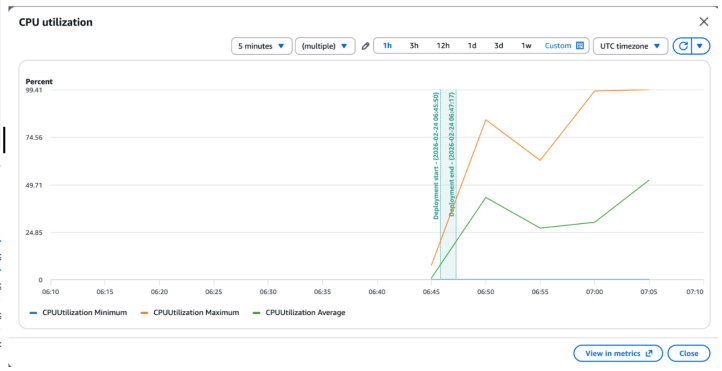
- Analysis: Horizontal scaling distributed the load across 2 tasks, effectively eliminating the context-switching bottleneck and keeping tail latency stable.

Test 2: Auto Scaling Trigger (400 Users)

Goal: Verify dynamic capacity adjustment under heavy load.

- Initial State: 2 Tasks.
- Peak State: 4 Tasks.
- Scaling Evidence: CloudWatch Average CPU reached ~50%, triggering the target tracking policy to scale from 2 to 4 tasks. Maximum CPU hit near 100%, justifying the need for additional instances.

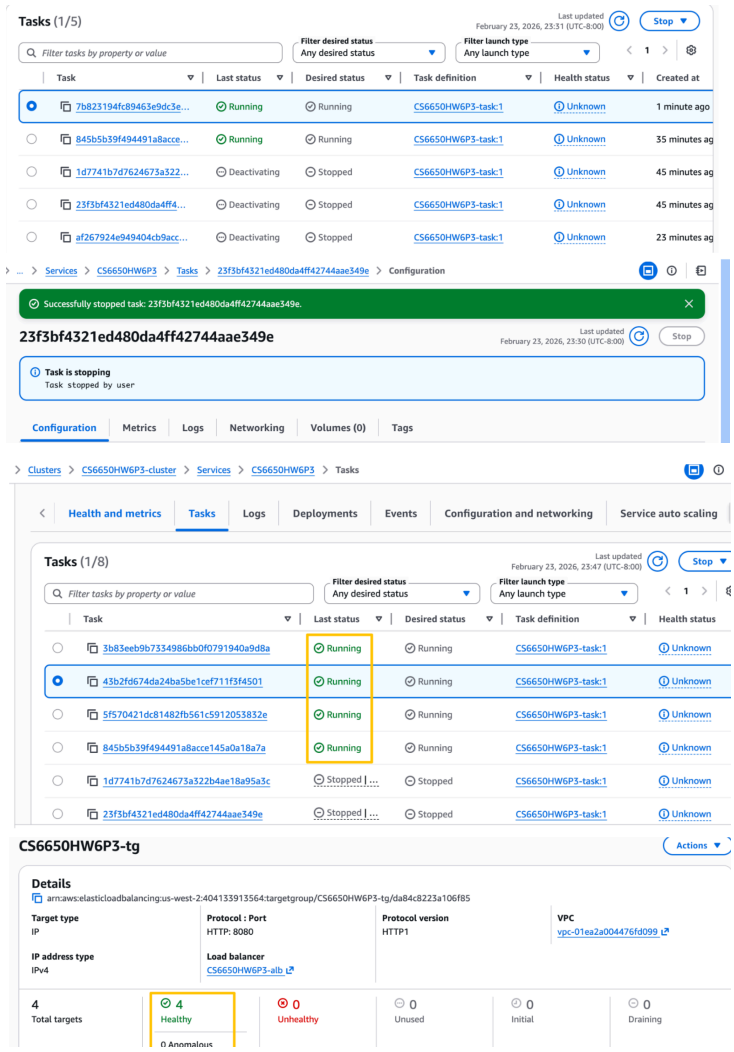




Test 3: Resilience & Fault Tolerance

Goal: Ensure zero-downtime during instance failure.

- Action: Manually stopped 1 task during load.
- Observation: ALB redirected traffic via Target Group health checks.
- Result: 0% Failure Rate. ECS self-healed by provisioning a new replacement task.

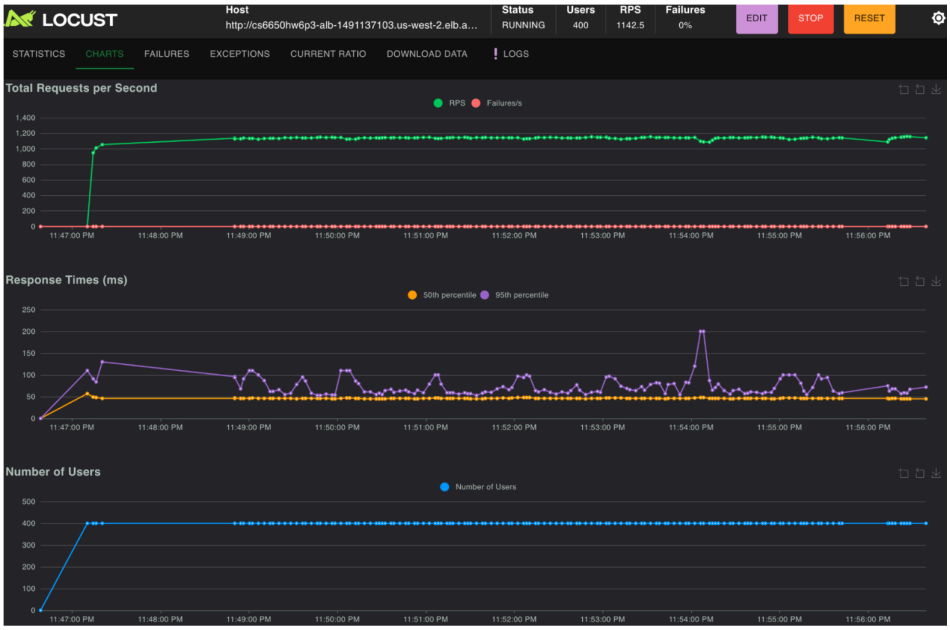


Test 4: Load Pattern Predictions

Goal: Compare system response to sudden vs. gradual traffic increases.

- Sudden Spike (-u 400, -r 400):

-u 400 -r 400 --run-time 5m



- Observation: RPS jumped immediately to ~1142.
- Result: High initial stress on existing tasks; 95th percentile latency showed significant jitter (spiking up to 200ms) during the first few minutes of the burst.
- Conclusion: High ramp rates challenge the system's "cold start" response time.

- Gradual Ramp (-u 400, -r 1):

-u 400 -r 1 --run-time 10m

- Observation: RPS increased linearly over 10 minutes.
- Result: Extremely smooth performance. Once the system reached the 400-user steady state (~1143 RPS), 95th percentile latency stabilized beautifully below 80ms.

- Conclusion: Auto Scaling performs optimally with predictable, gradual load increases.

Architectural Discussion

1. Component Roles

- ALB: Acts as the traffic conductor, balancing load and masking instance failures.
- Target Group: Orchestrates task registration and monitors health via /health.
- Auto Scaling: Provides elasticity, matching compute supply to real-time CPU demand.

2. Trade-offs: Horizontal vs. Vertical Scaling

Feature	Horizontal (Used)	Vertical (256 → 512 CPU)
Availability	High (Redundancy)	Low (Single Point of Failure)
Ceiling	Virtually Unlimited	Hard Hardware Limit
Complexity	Higher (Requires ALB)	Minimal (Config change)

Conclusion

Horizontal scaling proved to be the definitive solution for our compute-bound service. By distributing load and automating capacity management, we successfully increased throughput by 232% (from 344 to 1143 RPS) while maintaining a significantly better latency profile than a single-instance setup.